OCS INTELLIGENCE · SPRINT 1 · PLAYBOOK

Multi-service llama.cpp API playbook

The live path. Admission gateway, orchestrator, and two llama-server workers.

STATE **done** | DATE 2026-09-30 | SOURCE evidence/sprint1/2026-09-30-multi-service-llama-cpp-api.md

Read the first half to explain the live services. Use the second half to check which hop failed. This playbook covers only what is built: the EC2 admission gateway, the rig orchestrator, and the two llama-server workers. Nginx is the TLS front door in front of that path. It is not an inference service.

A FastAPI broker, Redis, RDS, per-student keys, and Rig 2 appear in the Week 0+1 mentor deck. They are not running. They are listed at the end so they are not mistaken for the live path.

> **NOTE**
>
> Written 2026-09-30 and re-rendered on 2026-10-16 in the standard playbook format. The wording is unchanged, so this is a record of 2026-09-30. Some of it has changed since: NetBird now limits rig `:9000` to EC2 and operator peers, and Rig 2 is serving. For the current state, see status.

The live path

A student points GitHub Copilot Chat, or curl, at `https://ai.opencodingsociety.com/v1`. Tokens stream back as Server-Sent Events. No hop buffers the stream.

HOP	WHERE	UNIT / PROCESS	JOB
1	EC2 `:443`	Nginx (`llm-relay`)	TLS. `/v1/` and `/healthz` go to `localhost:9100`. `/` goes to Open WebUI, not this API.
2	EC2 `127.0.0.1:9100`	`ocs-gateway`	Check the student key. Hold extra clients in a per-model wait line. Proxy the ones that fit.
3	Rig `0.0.0.0:9000`	`ocs-orchestrator`	Check the key again. Route by model name. Swap in the worker secret. Pass SSE through.
4a	Rig `127.0.0.1:8081`	`ocs-llama-a`	llama-server. `qwen3.8:27b` on GPUs 0-4.
4b	Rig `127.0.0.1:8082`	`ocs-llama-b`	llama-server. `qwen2.5:0.5b` on GPUs 5-6.

The rig is reachable only over NetBird (`100.75.123.203`). EC2 is `100.75.172.167`. The public name `ai.opencodingsociety.com` resolves to EC2. Workers bind to `127.0.0.1`, so they are not on the overlay.

> **NOTE**
>
> Open WebUI on rig port 3000 is the browser chat at `/`. It does not pass through the gateway or the orchestrator. A failure in Copilot is not diagnosed by looking at Open WebUI.

Why the work is split

Rig 1 is an ASUS B250 Mining Expert with seven GTX 1070s (8 GB, Pascal, `sm_61`) on PCIe Gen1 x1 risers. There is no NVLink. Spanning one model across all seven cards costs six inter-GPU hops per token. The live split is 5+2: Worker A uses GPUs 0-4 (four hops), Worker B uses GPUs 5-6 (one hop).

- On the public path, waiting happens on EC2. The rig then only receives work the gateway has already admitted. Rig port 9000 on NetBird can still bypass that line.
- Clients speak OpenAI Chat Completions. Failures are 400 (prompt longer than the window), 401, 404, 429, or 503.
- Each process has one job. Nginx terminates TLS. The gateway admits. The orchestrator routes. llama-server generates.

CUDA 13 cannot compile for `sm_61`. The workers run a llama.cpp build made with CUDA 12.6 (recorded build `0.4.1-dev`, commit `972d231`). The binary's RUNPATH points at `/opt/llama.cpp/src/build/bin`. Deleting that build directory breaks the workers. The build has no NCCL. Worker A is slower than the old all-GPU Ollama baseline; that is still an open issue, not a gateway bug.

Each service

Admission gateway (EC2)

FastAPI on `127.0.0.1:9100`, one Uvicorn worker. The wait line is in-process memory. A second Uvicorn worker would be a second line that cannot see the first. Code: `gateway/main.py`, `gateway/lanes.py`, `gateway/auth.py`.

Two lanes, one per model. They do not share slots. Defaults from `gateway/config.py`, confirmed live on 2026-09-18:

LANE	MODEL	IN FLIGHT	WAITING	NEXT REQUEST
A	`qwen3.8:27b`	1	4	6th concurrent request is HTTP 429
B	`qwen2.5:0.5b`	2	4	7th concurrent request is HTTP 429

- The Bearer key is checked before a client can join the line. A bad key is 401 and does not occupy a slot.
- An unknown model is 404 and is not queued.
- `/v1/models` and `/healthz` do not take a GPU slot.
- There is no wait timeout. Streaming clients may see SSE comments (`: queued`) about every 15 seconds.
- Nginx `proxy_read_timeout` on `/v1/` is 3600 seconds so a long 27B wait is not cut at the proxy.
- Closing the client drops that client's place. It does not hold a GPU for a disconnected tab.

The gateway does not change llama-server `--parallel`, and it does not cap `max_tokens` or context. A long 27B generation holds the only quality slot until it finishes.

Orchestrator (rig)

FastAPI on `0.0.0.0:9000` so EC2 can reach it over NetBird. It is a second auth check: anyone who talks to `:9000` directly, skipping the gateway, is still checked against `PUBLIC_API_KEYS`. There is no firewall rule yet that limits `:9000` to the EC2 peer. That direct path also skips the wait line.

- `qwen3.8:27b` routes to `http://127.0.0.1:8081`. `qwen2.5:0.5b` routes to `http://127.0.0.1:8082`.
- The orchestrator replaces the student key with `WORKER_API_KEY` before it calls llama-server. Students never see that secret.
- It polls each worker's `/health`. Status values `ok`, `loading model`, and `no slot available` all count as healthy. Anything else, or no answer, is unhealthy and the model returns 503.
- `QUEUE_CAPACITY_PER_WORKER` (16) is a reject threshold, not a wait line. The public path does not reach it, because the gateway admits fewer requests than that.
- There is one worker per model. The orchestrator does not load-balance across replicas.

School WiFi blocks PyPI, so the orchestrator venv uses Ubuntu packages (`python3-fastapi`, `python3-uvicorn`, `python3-httpx`) via `--system-site-packages`.

Worker A and Worker B (rig)

Both are systemd units that exec `/opt/llama.cpp/bin/llama-server`. Weights stay on the GPU (`--n-gpu-layers 999`) and are split by layer across the group's cards. `GGML_CUDA_P2P` is unset. Do not turn peer-to-peer on until P2P tests pass for every pair.

	WORKER A	WORKER B
Unit	`ocs-llama-a`	`ocs-llama-b`
Model id	`qwen3.8:27b`	`qwen2.5:0.5b`
File	`qwen3.8-27b-q4_k_m.gguf` (~16.8 GB)	`qwen2.5-0.5b.gguf` (~380 MB)
GPUs	0, 1, 2, 3, 4	5, 6
Bind	`127.0.0.1:8081`	`127.0.0.1:8082`
`--ctx-size`	262144 (`CONTEXT_SIZE`)	8192 (`CONTEXT_SIZE`)
`--parallel`	1	2
Window per request	262,144 tokens	4,096 tokens
Role	Quality. This model thinks before it answers.	Fast short answers.

llama-server divides `--ctx-size` by `--parallel` when the slot count is set explicitly. That was read from Worker A's own log on the previous setting: `CONTEXT_SIZE=4096` and `--parallel 2` reported `n_slots = 2` and `n_ctx_slot = 2048`. Worker B was not re-read on 2026-09-23. The same rule, applied to its unchanged 8192 context and `--parallel 2`, gives a 4,096-token window per request. Prompt tokens and generated tokens share that window. `max_tokens` only caps the reply. It does not enlarge the window.

On 2026-09-23 Worker A was changed from 4096 / two slots to 262144 / one slot, which is the model's own maximum and the largest power of two that stayed on the five 1070s with an f16 KV cache. `GET /props` on Worker A reported `n_ctx` 262144 and `total_slots` 1. A ~3,000-token prompt was accepted. A 400,052-token prompt was rejected with `exceed_context_size_error`. Startup logged one compute-buffer allocation failure on GPU 2, then llama.cpp retried without pipeline parallelism and came up. After a few-thousand-token prompt, GPU 2 was about 7.4 GB of 8 GB. A prompt near the full window has little scratch memory left and will be slow to prefill on the x1 risers.

> **WARNING**
>
> `docs/architecture/workers.md` (last verified 2026-09-18) still says Worker A context 4096 and `--parallel 2`. That page is older than the live unit. The repo unit `infra/rig/ocs-llama-a.service` has `--parallel 1`. The measured window is written in `CONTEXT.md` and `STATUS.md` section 4.2. Trust those, not the 4096 line in `workers.md`, until that page is updated.

`qwen3.8:27b` emits reasoning before the visible answer. A small `max_tokens` can be spent entirely on `reasoning_content` and the visible content comes back empty. Use 80 or more for a short answer. Measured generation, 2026-09-17 through 2026-09-18: Worker A about 8.6 to 8.7 tokens/s; Worker B about 80 to 97 tokens/s.

This llama.cpp build requires `--flash-attn on` (a value, not a bare flag). A bare `--flash-attn` once swallowed the next argument and crash-looped the unit.

What a request decides

Order on `POST /v1/chat/completions`, public path:

1. Nginx sends `/v1/` to the gateway. It does not rewrite the URI.
2. Gateway: missing or wrong Bearer key → 401. Unknown model → 404. Lane full (in flight + waiters) → 429 with `Retry-After: 15`. Otherwise the request waits, if needed, then is proxied to the orchestrator.
3. Orchestrator: wrong key → 401. Unknown model → 404. Worker unhealthy → 503. Then it calls the matching llama-server with the worker key.
4. llama-server: prompt longer than that worker's per-request window → 400 `exceed_context_size_error`. Otherwise it generates until stop or `max_tokens`.

The orchestrator can also return 429 if its own reject threshold of 16 is hit. On the public path the gateway stops the client earlier, so that 429 is the one you see when something bypasses EC2 and talks to rig `:9000`.

How to check it

Start order: workers, then orchestrator, then gateway. Worker A can take minutes to load the 27B weights. The runbook marks these procedures as not yet drilled. The commands below are the ones the units and the runbook specify. Do not print API keys.

Which hop is down

CHECK	PASS LOOKS LIKE	IF IT FAILS
`curl -sS https://ai.opencodingsociety.com/healthz`	Gateway process is up.	Nginx or `ocs-gateway` on EC2. The rig can still be fine.
`GET /v1/models` with the student key	Both model ids listed.	401: key. 200 with one model: orchestrator config or a worker the orchestrator has not registered.
On the rig: `systemctl is-active ocs-llama-a ocs-llama-b ocs-orchestrator`	`active` three times.	The named unit. Read `journalctl -u <unit> -n 80`.
`curl -s http://127.0.0.1:9000/readyz`	Status `ready`.	503: neither worker is healthy. Check llama-server `/health` and `nvidia-smi`.
`nvidia-smi` memory	GPUs 0-4 hold Worker A. GPUs 5-6 hold Worker B.	A card near 8 GB on Worker A during a long prompt can fail the compute buffer. That is the 262144-token fit, not a routing bug.

One completion

From a checkout with `OCS_API_KEY` in `.env`. This hits the public path, so it also tests the wait line:

```
set -a && source .env && set +a
curl -sS "${OCS_BASE_URL:-https://ai.opencodingsociety.com/v1}/chat/completions" \
  -H "Authorization: Bearer $OCS_API_KEY" \
  -H "Content-Type: application/json" \
  -d '{"model":"qwen3.8:27b","messages":[{"role":"user","content":"In one sentence, what is a binary search?"}],"max_tokens":80}'
```

Expect several seconds, then a JSON chat completion. HTTP 429 means the single 27B slot and its four waiters are full. Retry after the `Retry-After` value. An empty `content` field with a large `reasoning_content` means `max_tokens` was spent on thinking.

Restart one model

Each model has one worker. Restarting `ocs-llama-a` makes `qwen3.8:27b` return 503 until the weights load again. Do it outside class, or tell users first. Worker B can stay up. Do not restart the orchestrator or the gateway unless their own config changed.

```
sudo systemctl restart ocs-llama-a
# wait until curl -s http://127.0.0.1:8081/health returns ok
# confirm n_ctx with an authenticated GET /props: 262144, total_slots 1
```

Open items

Not built as of 2026-09-30:

- Broker that picks among many workers (healthy, capable, already loaded, least busy) with Redis and RDS.
- Per-student API keys, quotas, and revocation. One shared student key today.
- Rig 2. This playbook is Rig 1 only.
- Prometheus, GPU exporter, and Grafana. llama-server already exposes `/metrics` and `/slots` for that later.
- A firewall so only EC2 can open rig port 9000.
- A fix for Worker A being slower than the old 7-GPU Ollama baseline (~8.7 tok/s vs ~12.5).

Sources

Written 2026-09-30 from the repo. Context-window numbers were measured on the rig on 2026-09-23 (Worker A `/props`, a short completion, a ~3,000-token accept, a 400,052-token reject, `nvidia-smi`). Lane sizes and token speeds were measured 2026-09-17 and 2026-09-18. This PDF was not a new live drill.

- `docs/architecture/overview.md`, `gateway.md`, `orchestrator.md`
- `docs/reference/hosts.md` and the units in `infra/rig/` and `infra/ec2/`
- `gateway/config.py`, `orchestrator/routing.py`, `orchestrator/health.py`
- `CONTEXT.md` and `STATUS.md` section 4.2 for the 262,144-token window
- `2026-09-17-public-api-e2e.md` and `2026-09-18-gateway-concurrency.md`

Do not put student keys, worker keys, or NetBird SSO codes in a copy of this playbook.